

Guida completa al codice: Gaussian Mixture Model con algoritmo EM

Midterm 1 — Graphical and Deep Learning

Appunti di studio

Anno Accademico 2025–2026

Indice

1	Contesto teorico	3
1.1	Perché i dati sono “incompleti”?	3
1.2	Panoramica del flusso	3
2	Il modello GMM	3
2.1	Definizione formale	3
2.2	Gaussiana multivariata con covarianza diagonale	4
2.3	Parametri nel codice e loro shape	5
3	Incomplete vs Complete Likelihood	5
3.1	Incomplete (Marginal) Likelihood	5
3.2	Complete Likelihood	6
3.3	Il metodo <code>log_likelihood</code> nel codice	6
4	<code>_compute_log_weighted</code>: dalla formula al codice	7
4.1	Espansione della log-gaussiana	7
4.2	Il codice completo	8
4.3	Analisi riga per riga	9
4.3.1	Riga 2: dimensionalità	9
4.3.2	Riga 3: differenze con broadcasting	9
4.3.3	Righe 4–5: distanza di Mahalanobis scalata	9
4.3.4	Riga 6: somma sulle features — Termine C	10
4.3.5	Riga 7: log-determinante — Termine B	10
4.3.6	Righe 8–10: assemblaggio della log-gaussiana	10
4.3.7	Riga 11: aggiunta dei pesi	10
4.4	Tabella riepilogativa delle shape	11
5	Log-Sum-Exp Trick	11
5.1	Il problema numerico	11
5.2	La soluzione matematica	11
5.3	Esempio numerico	11
5.4	Corrispondenza nel codice	12
6	K-Means++ Initialization	12
6.1	Perché è necessaria una buona inizializzazione	12
6.2	L'algoritmo K-Means++	13
6.3	Il codice	13

7	Inizializzazione di pi, mu, sigma in fit	13
7.1	Medie: K-Means++	14
7.2	Pesi: uniformi	14
7.3	Varianze: varianza globale replicata	14
8	Algoritmo EM	14
8.1	Schema generale (dalle slide)	14
8.2	E-step: calcolo delle responsabilità	16
8.3	M-step: aggiornamento dei parametri	17
8.3.1	Riga 2: conteggio soft N_k	17
8.3.2	Riga 3: floor su N_k	17
8.3.3	Riga 4: aggiornamento pesi π_k	18
8.3.4	Riga 5: aggiornamento medie μ_k	18
8.3.5	Righe 6–8: aggiornamento varianze σ_k^2	18
8.3.6	Riga 9: floor sulle varianze	18
8.4	Convergenza	19
9	BIC (Bayesian Information Criterion)	19
9.1	Formula	19
9.2	Conteggio dei parametri	19
9.3	Il codice	20
9.4	Risultati del training e interpretazione	20
10	Checklist di verifica	21
10.1	Proprietà da verificare	21
10.2	Tabella riassuntiva	22

1 Contesto teorico

Il corso di *Graphical and Deep Learning* (GDL) organizza il problema dell'apprendimento nei modelli grafici secondo due assi ortogonali (GDL7, slide 9, GDL8, slide 3):

	Complete Data	Incomplete Data
Known Structure	MLE diretto	EM Algorithm
Unknown Structure	Structure Learning	Structural EM

Posizione del midterm nella tassonomia

Il nostro midterm si colloca nel quadrante **Fixed (Known) Structure + Incomplete Data**. La struttura del modello grafico è fissata (un GMM con K componenti), ma i dati sono *incompleti* perché non osserviamo le variabili latenti z_j che indicano a quale componente appartiene ogni campione x_j . L'algoritmo per eccellenza in questo quadrante è l'**Expectation-Maximization (EM)**.

1.1 Perché i dati sono “incompleti”?

In un Gaussian Mixture Model, ogni campione x_j è stato generato da *una* delle K componenti gaussiane, ma non sappiamo quale. Se conoscessimo l'assegnamento $z_j \in \{1, \dots, K\}$ per ogni campione, potremmo stimare i parametri con semplice MLE (quadrante in alto a sinistra). Poiché z_j è *latente*, dobbiamo usare EM.

1.2 Panoramica del flusso

Il codice che abbiamo scritto segue esattamente questo schema:

1. **Inizializzazione:** K-Means++ per le medie, varianza globale per le covarianze, pesi uniformi.
2. **E-step:** calcolo delle responsabilità γ_{jk} (posterior sulle variabili latenti).
3. **M-step:** aggiornamento di π_k , μ_k , Σ_k usando le responsabilità.
4. **Convergenza:** controllo sulla log-likelihood.
5. **Model selection:** BIC per scegliere il numero ottimale di componenti.

2 Il modello GMM

2.1 Definizione formale

Un *Gaussian Mixture Model* con K componenti definisce la densità di probabilità di un vettore $x \in \mathbb{R}^d$ come (GDL8, slide 19–20):

$$p(x \mid \theta) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k) \quad (1)$$

dove $\theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$ è l'insieme dei parametri e:

- $\pi_k \geq 0$ sono i **mixing coefficients** (pesi), con $\sum_k \pi_k = 1$.
- $\mu_k \in \mathbb{R}^d$ è il **vettore medio** della k -esima componente.
- Σ_k è la **matrice di covarianza** della k -esima componente.

2.2 Gaussian multivariata con covarianza diagonale

La slide 18 di GDL8 mostra la gaussiana **univariata** ($d = 1$, un singolo numero reale):

$$\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi} \sigma} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right) \quad (2)$$

Questa formula ha tre pezzi: una costante $\frac{1}{\sqrt{2\pi}}$, un fattore di scala $\frac{1}{\sigma}$ che dipende dalla “larghezza” della campana, e l’esponenziale che misura quanto x è lontano da μ .

La **formula multivariata** (d dimensioni) generalizza ciascuno di questi tre pezzi:

	Univariata ($d = 1$)	Multivariata (d dim.)	Perché
Costante	$\frac{1}{\sqrt{2\pi}}$	$\frac{1}{(2\pi)^{d/2}}$	una copia di $\sqrt{2\pi}$ per ogni dimensione
Scala	$\frac{1}{\sigma}$	$\frac{1}{ \Sigma ^{1/2}}$	$\sigma \rightarrow$ radice del determinante della matrice
Distanza	$\frac{(x - \mu)^2}{\sigma^2}$	$(x - \mu)^\top \Sigma^{-1} (x - \mu)$	distanza di Mahalanobis (tiene conto delle correlazioni)

Sostituendo nella stessa struttura della (2) si ottiene la formula generale:

$$\mathcal{N}(x \mid \mu, \Sigma) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1} (x - \mu)\right) \quad (3)$$

Se poni $d = 1$, $\Sigma = \sigma^2$ (scalare), $|\Sigma|^{1/2} = \sigma$ e $(x - \mu)^\top \Sigma^{-1} (x - \mu) = (x - \mu)^2 / \sigma^2$: si riottiene esattamente la (2) della slide 18.

Perché $p(x_j \mid \mu_k, \Sigma_k) = \mathcal{N}(x_j \mid \mu_k, \Sigma_k)$?

Questa uguaglianza *non* è un risultato da dimostrare: è una **scelta di modello**. Quando definiamo un GMM, *assumiamo* che ogni componente k generi i dati secondo una distribuzione gaussiana. Pertanto la densità di probabilità di osservare il campione x_j , sapendo che proviene dalla componente k , è *per definizione* la funzione di densità gaussiana:

$$p(x_j \mid z_j = k, \theta) = \mathcal{N}(x_j \mid \mu_k, \Sigma_k)$$

In altre parole, $\mathcal{N}(\cdot)$ è semplicemente il *nome* che diamo alla funzione di densità quando ha la forma a campana definita dalla formula (3). Scrivere $\mathcal{N}$ al posto di p è solo una notazione più compatta: le due scritture indicano la stessa quantità.

Nel nostro codice usiamo una **covarianza diagonale**: $\Sigma_k = \text{diag}(\sigma_{k,1}^2, \dots, \sigma_{k,d}^2)$. Questo semplifica enormemente i calcoli:

$$|\Sigma_k| = \prod_{i=1}^d \sigma_{k,i}^2 \quad (4)$$

$$(x - \mu_k)^\top \Sigma_k^{-1} (x - \mu_k) = \sum_{i=1}^d \frac{(x_i - \mu_{k,i})^2}{\sigma_{k,i}^2} \quad (5)$$

Covarianza diagonale: vantaggi pratici

Con covarianza diagonale:

- Non serve invertire matrici: basta dividere elemento per elemento.
- Il determinante è il prodotto delle varianze.
- La complessità scende da $O(d^3)$ a $O(d)$ per componente.
- In NumPy si memorizza come un vettore di shape $(n_categories, n_features)$, non come matrice $d \times d$.

2.3 Parametri nel codice e loro shape

Parametro	Attributo	Shape NumPy	Significato
π_k	<code>self._pi</code>	$(K,)$	Peso della componente k
μ_k	<code>self._mu</code>	(K, d)	Media della componente k
σ_k^2	<code>self._sigma</code>	(K, d)	Varianze diagonali della componente k

Nel nostro caso: $K \in \{1, \dots, 6\}$ (provati tutti), $d = 5$ features, $N = 800$ campioni dal file `train.csv`.

```
1 class GaussianMixtureModel:
2     def __init__(self, n_categories: int, n_features: int,
3                   debug: bool = False, plot: bool = False):
4         self.n_categories = n_categories
5         self._pi = None      # Shape: (n_categories,)
6         self._mu = None      # Shape: (n_categories, n_features)
7         self._sigma = None   # Shape: (n_categories, n_features)
8         self.max_iteration = 1000
9         self.epsilon = 1e-8
10        self.debug = debug
11        self.plot = plot
12        self._ll_history = []
```

Listing 1: Costruttore della classe

Perche usare None e non array vuoti

Inizializzare a None permette al metodo `fit()` di controllare se i parametri sono già stati impostati dall'utente (ad esempio per un warm-start) oppure se devono essere inizializzati automaticamente. È un pattern comune nelle librerie ML come `scikit-learn`.

3 Incomplete vs Complete Likelihood

Questa distinzione è **fondamentale** per capire perché serve EM e cosa calcola ogni pezzo del codice.

3.1 Incomplete (Marginal) Likelihood

La **incomplete likelihood** (GDL8, slide 21) è la probabilità dei dati osservati, marginalizzando sulle variabili latenti:

$$L(\theta | X) = \prod_{j=1}^N p(x_j | \theta) = \prod_{j=1}^N \sum_{k=1}^K \pi_k \mathcal{N}(x_j | \mu_k, \Sigma_k) \quad (6)$$

In forma logaritmica (la **incomplete log-likelihood**):

$$\ell(\theta | X) = \sum_{j=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(x_j | \mu_k, \Sigma_k) \right) \quad (7)$$

Il logaritmo della somma non si semplifica

Nella (7) il logaritmo “avvolge” una somma. Non possiamo portare il log dentro la sommatoria. Per questo serve il **log-sum-exp trick** (Sezione 5) e per questo la MLE diretta è intrattabile — si usa EM.

3.2 Complete Likelihood

Se conoscessimo le variabili latenti z_j , avremmo la **complete likelihood** (GDL8, slide 24). Si introduce un indicatore $z_{jk} \in \{0, 1\}$ con $\sum_k z_{jk} = 1$ per ogni j :

$$L_c(\theta | X, Z) = \prod_{j=1}^N \prod_{k=1}^K [\pi_k \mathcal{N}(x_j | \mu_k, \Sigma_k)]^{z_{jk}} \quad (8)$$

La **complete log-likelihood**:

$$\ell_c(\theta | X, Z) = \sum_{j=1}^N \sum_{k=1}^K z_{jk} [\log \pi_k + \log \mathcal{N}(x_j | \mu_k, \Sigma_k)] \quad (9)$$

Come si collegano nel codice

- Il metodo `log_likelihood()` calcola la **incomplete log-likelihood** (7). È usato per il BIC e per il monitoraggio.
- L'E-step di EM ottimizza internamente la **expected complete log-likelihood**: sostituisce z_{jk} con il suo valore atteso γ_{jk} (le responsabilità).
- L'M-step massimizza questa quantità rispetto a θ .

3.3 Il metodo `log_likelihood` nel codice

```
1 def log_likelihood(self, samples: np.ndarray) -> np.ndarray:
2     log_w = self._compute_log_weighted(samples)
3     a_max = np.max(log_w, axis=1)
4     log_sum = a_max + np.log(
5         np.sum(np.exp(log_w - a_max[:, None]), axis=1)
6     )
7     return np.sum(log_sum)
```

Listing 2: Calcolo della incomplete log-likelihood

Corrispondenza riga per riga con la formula (7):

Codice	Formula
<code>log_w = self._compute_log_weighted(samples)</code>	$\log \pi_k \mathcal{N}(x_j \mu_k, \Sigma_k)$ per ogni j, k
<code>a_max = np.max(log_w, axis=1)</code>	$a_j = \max_k \log w_{jk}$ (per stabilità numerica)
<code>np.exp(log_w - a_max[:, None])</code>	$\exp(\log w_{jk} - a_j)$
<code>np.sum(..., axis=1)</code>	$\sum_k \exp(\log w_{jk} - a_j)$
<code>a_max + np.log(...)</code>	$a_j + \log \sum_k \exp(\log w_{jk} - a_j) = \log \sum_k w_{jk}$
<code>np.sum(log_sum)</code>	$\sum_j \log \sum_k w_{jk} = \ell(\theta X)$

Quindi `log_likelihood` restituisce esattamente la formula (7), calcolata in modo numericamente stabile grazie al log-sum-exp trick.

4 _compute_log_weighted: dalla formula al codice

Questa è la funzione *cuore* dell'implementazione. Calcola, per ogni campione j e ogni componente k :

$$\log w_{jk} = \log \pi_k + \log \mathcal{N}(x_j \mid \mu_k, \Sigma_k) \quad (10)$$

4.1 Espansione della log-gaussiana

Partendo dalla (3) con covarianza diagonale, il logaritmo della gaussiana si decompone in tre termini:

$$\log \mathcal{N}(x_j \mid \mu_k, \Sigma_k) = \underbrace{-\frac{d}{2} \log(2\pi)}_{\text{Termine A}} - \underbrace{\frac{1}{2} \sum_{i=1}^d \log \sigma_{k,i}^2}_{\text{Termine B}} - \underbrace{\frac{1}{2} \sum_{i=1}^d \frac{(x_{j,i} - \mu_{k,i})^2}{\sigma_{k,i}^2}}_{\text{Termine C}} \quad (11)$$

Derivazione passo-passo della formula ()

Passo 0 — riscrittura come prodotto. La formula (3) è già un prodotto di tre pezzi; li separiamo solo per chiarezza:

$$\underbrace{\frac{1}{(2\pi)^{d/2} |\Sigma_k|^{1/2}}}_{\text{denominatore della frazione}} = \underbrace{\frac{1}{(2\pi)^{d/2}}}_{\text{fattore 1}} \cdot \underbrace{\frac{1}{|\Sigma_k|^{1/2}}}_{\text{fattore 2}}$$

perché $\frac{1}{A \cdot B} = \frac{1}{A} \cdot \frac{1}{B}$. Quindi la (3) diventa:

$$\mathcal{N}(x_j | \mu_k, \Sigma_k) = \underbrace{\frac{1}{(2\pi)^{d/2}}}_{\text{fatt. 1}} \cdot \underbrace{\frac{1}{|\Sigma_k|^{1/2}}}_{\text{fatt. 2}} \cdot \underbrace{\exp\left(-\frac{1}{2}(x_j - \mu_k)^\top \Sigma_k^{-1} (x_j - \mu_k)\right)}_{\text{fatt. 3}}$$

Non abbiamo aggiunto né tolto nulla: è la stessa espressione scritta in forma più comoda per applicare il logaritmo.

Passo 1 — applichiamo log. Poiché $\log(a \cdot b \cdot c) = \log a + \log b + \log c$, il logaritmo del prodotto diventa una *somma* di tre logaritmi:

Fattore	Regola usata	Risultato
1	$\log((2\pi)^{-d/2}) = -\frac{d}{2} \log(2\pi)$ usando $\log(a^n) = n \log a$	Termine A
2	$\log(\Sigma_k ^{-1/2}) = -\frac{1}{2} \log \Sigma_k $ $= -\frac{1}{2} \log \prod_i \sigma_{k,i}^2 = -\frac{1}{2} \sum_i \log \sigma_{k,i}^2$ usando $\log \prod = \sum \log$ (e Σ_k diagonale, per cui $ \Sigma_k = \prod_i \sigma_{k,i}^2$ come in (4))	Termine B
3	$\log(\exp(y)) = y$ (log e exp si annullano); poi $(x_j - \mu_k)^\top \Sigma_k^{-1} (x_j - \mu_k) = \sum_i \frac{(x_{j,i} - \mu_{k,i})^2}{\sigma_{k,i}^2}$ usando la (5) (Σ_k diagonale)	Termine C

Sommando i tre risultati si ottiene esattamente la (11).

Verifica rapida con $d = 1$. Con una sola feature: Termine A = $-\frac{1}{2} \log(2\pi)$, Termine B = $-\frac{1}{2} \log \sigma_k^2 = -\log \sigma_k$, Termine C = $-\frac{(x_j - \mu_k)^2}{2\sigma_k^2}$. Sommando: $\log\left(\frac{1}{\sqrt{2\pi} \sigma_k} \exp\left(-\frac{(x_j - \mu_k)^2}{2\sigma_k^2}\right)\right)$, ossia il logaritmo della gaussiana univariata della slide 18.

4.2 Il codice completo

```

1 def _compute_log_weighted(self, samples: np.ndarray):
2     d = samples.shape[1]
3     differences = samples[:, None, :] - self._mu[None, :, :]
4     differences_squares = differences ** 2
5     scaled = differences_squares / self._sigma[None, :, :]
6     third_term = np.sum(scaled, axis=2)
7     second_term_log_var_sum = np.sum(np.log(self._sigma), axis=1)

```

```

8     log_gauss = (-0.5 * d * np.log(2 * np.pi)
9                 - 0.5 * second_term_log_var_sum[None, :])
10                - 0.5 * third_term)
11     log_weighted = log_gauss + np.log(self._pi)[None, :]
12     return log_weighted

```

Listing 3: `_compute_log_weighted`: il cuore del GMM

4.3 Analisi riga per riga

4.3.1 Riga 2: dimensionalità

```
1 d = samples.shape[1] # d = 5 nel nostro caso
```

Semplicemente il numero di features. `samples` ha shape $(N, d) = (800, 5)$.

4.3.2 Riga 3: differenze con broadcasting

```
1 differences = samples[:, None, :] - self._mu[None, :, :]
```

Questa è la riga più importante da capire. Il broadcasting di NumPy calcola $x_{j,i} - \mu_{k,i}$ per *tutte* le coppie (j, k) simultaneamente:

Operando	Shape	Significato degli assi
<code>samples[:, None, :]</code>	$(N, 1, d)$	asse 0 = campione j , asse 1 = broadcast, asse 2 = feature i
<code>self._mu[None, :, :]</code>	$(1, K, d)$	asse 0 = broadcast, asse 1 = componente k , asse 2 = feature i
<code>differences</code>	(N, K, d)	<code>differences[j, k, i]</code> = $x_{j,i} - \mu_{k,i}$

Esempio concreto di broadcasting

Con $N = 800$, $K = 4$, $d = 5$:

- `samples[:, None, :]` ha shape $(800, 1, 5)$ — ogni campione viene “ripetuto” per K componenti.
- `self._mu[None, :, :]` ha shape $(1, 4, 5)$ — ogni media viene “ripetuta” per N campioni.
- Il risultato ha shape $(800, 4, 5)$ — tutte le $800 \times 4 = 3200$ differenze, ciascuna un vettore di 5 elementi.

4.3.3 Righe 4–5: distanza di Mahalanobis scalata

```

1 differences_squares = differences ** 2
2 scaled = differences_squares / self._sigma[None, :, :]

```

Variabile	Shape input	Operazione	Shape output
<code>differences_squares</code>	(N, K, d)	$(\cdot)^2$	(N, K, d)
<code>self._sigma[None, :, :]</code>	$(1, K, d)$	broadcast	—
<code>scaled</code>	$(N, K, d) \div (1, K, d)$	$\frac{(x_{j,i} - \mu_{k,i})^2}{\sigma_{k,i}^2}$	(N, K, d)

4.3.4 Riga 6: somma sulle features — Termine C

```
1 third_term = np.sum(scaled, axis=2) # Shape: (N, K)
```

Calcola:

$$\text{third_term}[j, k] = \sum_{i=1}^d \frac{(x_{j,i} - \mu_{k,i})^2}{\sigma_{k,i}^2}$$

Questa è la distanza di Mahalanobis al quadrato (con covarianza diagonale). La somma su `axis=2` collassa la dimensione delle features, passando da shape (N, K, d) a (N, K) .

4.3.5 Riga 7: log-determinante — Termine B

```
1 second_term_log_var_sum = np.sum(np.log(self._sigma), axis=1)
```

Calcola per ogni componente k :

$$\text{second_term_log_var_sum}[k] = \sum_{i=1}^d \log \sigma_{k,i}^2 = \log \prod_{i=1}^d \sigma_{k,i}^2 = \log |\Sigma_k|$$

Shape: $(K,)$. Dalla (4), il determinante di una matrice diagonale è il prodotto degli elementi diagonali, quindi la somma dei logaritmi è il logaritmo del determinante.

4.3.6 Righe 8–10: assemblaggio della log-gaussiana

```
1 log_gauss = (-0.5 * d * np.log(2 * np.pi) # Termine A
2             - 0.5 * second_term_log_var_sum[None, :] # Termine B
3             - 0.5 * third_term) # Termine C
```

Termine	Shape	Broadcasting	Formula
A: $-0.5 \cdot d \cdot \text{np.log}(2 \cdot \text{np.pi})$	scalare	$\rightarrow (N, K)$	$-\frac{d}{2} \log(2\pi)$
B: $-0.5 \cdot \dots [\text{None}, :]$	$(1, K)$	$\rightarrow (N, K)$	$-\frac{1}{2} \log \Sigma_k $
C: $-0.5 \cdot \text{third_term}$	(N, K)	—	$-\frac{1}{2} \sum_i \frac{(x_{j,i} - \mu_{k,i})^2}{\sigma_{k,i}^2}$
<code>log_gauss</code>	(N, K)	—	$\log \mathcal{N}(x_j \mu_k, \Sigma_k)$

4.3.7 Riga 11: aggiunta dei pesi

```
1 log_weighted = log_gauss + np.log(self._pi)[None, :]
```

Operando	Shape	Formula
<code>log_gauss</code>	(N, K)	$\log \mathcal{N}(x_j \mu_k, \Sigma_k)$
<code>np.log(self._pi)[None, :]</code>	$(1, K)$	$\log \pi_k$ (broadcast su N)
<code>log_weighted</code>	(N, K)	$\log \pi_k + \log \mathcal{N}(x_j \mu_k, \Sigma_k)$

Il risultato finale è:

$$\text{log_weighted}[j, k] = \log(\pi_k \mathcal{N}(x_j | \mu_k, \Sigma_k))$$

4.4 Tabella riepilogativa delle shape

Variabile	Shape	Contenuto matematico
<code>samples</code>	(N, d)	Dati di input x_j
<code>samples[:, None, :]</code>	$(N, 1, d)$	x_j con asse per broadcast
<code>self._mu[None, :, :]</code>	$(1, K, d)$	μ_k con asse per broadcast
<code>differences</code>	(N, K, d)	$x_{j,i} - \mu_{k,i}$
<code>differences_squares</code>	(N, K, d)	$(x_{j,i} - \mu_{k,i})^2$
<code>self._sigma[None, :, :]</code>	$(1, K, d)$	$\sigma_{k,i}^2$ per broadcast
<code>scaled</code>	(N, K, d)	$(x_{j,i} - \mu_{k,i})^2 / \sigma_{k,i}^2$
<code>third_term</code>	(N, K)	$\sum_i (x_{j,i} - \mu_{k,i})^2 / \sigma_{k,i}^2$
<code>second_term_log_var_sum</code>	$(K,)$	$\sum_i \log \sigma_{k,i}^2$
<code>log_gauss</code>	(N, K)	$\log \mathcal{N}(x_j \mu_k, \Sigma_k)$
<code>np.log(self._pi)</code>	$(K,)$	$\log \pi_k$
<code>log_weighted</code>	(N, K)	$\log(\pi_k \cdot \mathcal{N}(x_j \mu_k, \Sigma_k))$

5 Log-Sum-Exp Trick

5.1 Il problema numerico

Dobbiamo calcolare:

$$\log \sum_{k=1}^K w_{jk} \quad \text{dove} \quad w_{jk} = \pi_k \mathcal{N}(x_j | \mu_k, \Sigma_k)$$

Il calcolo diretto richiede di esponenziare i `log_weightedjk` per ottenere w_{jk} , sommarli, e poi prendere il logaritmo. Ma $\log w_{jk}$ può essere molto negativo (es. -500), rendendo $w_{jk} = e^{-500} \approx 0$ in floating point (underflow).

5.2 La soluzione matematica

Il trucco si basa sull'identità:

$$\log \sum_k e^{a_k} = a^* + \log \sum_k e^{a_k - a^*} \quad \text{dove} \quad a^* = \max_k a_k \quad (12)$$

Dimostrazione:

$$\log \sum_k e^{a_k} = \log \left(e^{a^*} \sum_k e^{a_k - a^*} \right) = a^* + \log \sum_k e^{a_k - a^*}$$

Perche funziona

Dopo la sottrazione di a^* , tutti gli esponenti $a_k - a^*$ sono ≤ 0 . Il termine con $a_k = a^*$ contribuisce $e^0 = 1$, quindi la somma è ≥ 1 e il logaritmo è ≥ 0 . Nessun underflow!

5.3 Esempio numerico

Supponiamo $K = 3$ e per un certo campione j :

$$\log w_{j,1} = -502, \quad \log w_{j,2} = -500, \quad \log w_{j,3} = -505$$

Calcolo diretto (FALLISCE):

$$e^{-502} + e^{-500} + e^{-505} \approx 0 + 0 + 0 = 0 \quad \Rightarrow \quad \log(0) = -\infty \quad (\text{errore!})$$

Con **log-sum-exp** (FUNZIONA):

$$\begin{aligned}
 a^* &= \max(-502, -500, -505) = -500 \\
 &- 500 + \log\left(e^{-502-(-500)} + e^{-500-(-500)} + e^{-505-(-500)}\right) \\
 &= -500 + \log(e^{-2} + e^0 + e^{-5}) \\
 &= -500 + \log(0.1353 + 1 + 0.0067) \\
 &= -500 + \log(1.1420) \\
 &= -500 + 0.1328 = -499.867
 \end{aligned}$$

5.4 Corrispondenza nel codice

```

1 log_w = self._compute_log_weighted(samples) # (N, K)
2 a_max = np.max(log_w, axis=1) # (N,)
3 log_sum = a_max + np.log(
4     np.sum(np.exp(log_w - a_max[:, None]), axis=1)
5 ) # (N,)
6 return np.sum(log_sum) # scalare

```

Listing 4: Log-sum-exp in `log_likelihood`

Codice	Formula	Shape
<code>log_w</code>	$a_{jk} = \log w_{jk}$	(N, K)
<code>a_max</code>	$a_j^* = \max_k a_{jk}$	$(N,)$
<code>log_w - a_max[:, None]</code>	$a_{jk} - a_j^*$	(N, K)
<code>np.exp(...)</code>	$e^{a_{jk} - a_j^*}$	(N, K)
<code>np.sum(..., axis=1)</code>	$\sum_k e^{a_{jk} - a_j^*}$	$(N,)$
<code>a_max + np.log(...)</code>	$a_j^* + \log \sum_k e^{a_{jk} - a_j^*}$	$(N,)$
<code>np.sum(log_sum)</code>	$\sum_j \log \sum_k w_{jk}$	scalare

Lo stesso pattern si ripete nel fit

Il log-sum-exp trick appare *due volte* nel codice:

1. In `log_likelihood()`: per calcolare $\ell(\theta|X)$.
2. Nell'E-step di `fit()`: per calcolare il denominatore delle responsabilità γ_{jk} (stesse righe identiche).

6 K-Means++ Initialization

6.1 Perché è necessaria una buona inizializzazione

L'algoritmo EM è garantito convergere a un **massimo locale** della likelihood (GDL8, slide 15), ma non necessariamente al massimo globale. Il grafico a slide 15 mostra come EM cresca monotonicamente ma possa convergere a soluzioni diverse a seconda del punto di partenza.

Una cattiva inizializzazione può portare a:

- Componenti “vuote” (nessun campione assegnato).
- Componenti sovrapposte (ridondanti).
- Convergenza lenta.
- Log-likelihood finale bassa (massimo locale scarso).

6.2 L'algoritmo K-Means++

K-Means++ sceglie i centroidi iniziali in modo che siano ben distanziati tra loro:

Algorithm 1 K-Means++ Initialization

- 1: Scegli μ_1 uniformemente a caso tra i campioni
 - 2: **for** $k = 2, \dots, K$ **do**
 - 3: Per ogni campione x_j , calcola $D(x_j) = \min_{k' < k} \|x_j - \mu_{k'}\|^2$
 - 4: Scegli $\mu_k = x_j$ con probabilità proporzionale a $D(x_j)$
 - 5: **end for**
-

6.3 Il codice

```
1 def k_means_init(self, samples: np.ndarray):
2     sample_quantity, features_quantity = samples.shape
3     centroids = np.empty((self.n_categories, features_quantity))
4     centroids[0] = samples[(np.random.randint(sample_quantity))]
5     for k in range(1, self.n_categories):
6         distances_sq = np.array(
7             [np.sum((samples - mu_k_prime)**2, axis=1)
8              for mu_k_prime in centroids[:k]]
9         )
10        min_dist_sq = np.min(distances_sq, axis=0)
11        probabilities = min_dist_sq / np.sum(min_dist_sq)
12        next_index = np.random.choice(sample_quantity,
13                                     p=probabilities)
14        centroids[k] = samples[next_index]
15    return centroids
```

Listing 5: K-Means++ initialization

Analisi:

- **Riga 4:** il primo centroide è un campione scelto uniformemente a caso.
- **Righe 6–8:** per ogni centroide già scelto $\mu_{k'}$, calcola la distanza euclidea al quadrato da ogni campione. `distances_sq` ha shape (k, N) .
- **Riga 9:** per ogni campione, prende la distanza minima dal centroide più vicino. Shape: $(N,)$.
- **Riga 10:** normalizza per ottenere una distribuzione di probabilità. I campioni più lontani dai centroidi esistenti hanno probabilità maggiore di essere scelti.
- **Righe 11–12:** campiona il prossimo centroide secondo questa distribuzione.

Intuizione geometrica

K-Means++ garantisce che i centroidi iniziali siano “sparsi” nello spazio. Il primo è casuale; il secondo sarà probabilmente lontano dal primo; il terzo sarà lontano dai primi due; e così via. Questo evita il problema di inizializzare due componenti nello stesso cluster.

7 Inizializzazione di pi, mu, sigma in fit

```
1 def fit(self, samples: np.ndarray):
2     if self._mu is None:
3         self._mu = self.k_means_init(samples)
4     if self._pi is None:
```

```

5         self._pi = np.full(self.n_categories,
6                               1.0 / self.n_categories)
7     if self._sigma is None:
8         self._sigma = np.tile(np.var(samples, axis=0),
9                                (self.n_categories, 1))

```

Listing 6: Inizializzazione dei parametri nel metodo fit

7.1 Medie: K-Means++

Se `self._mu` non è stato pre-impostato, si usa K-Means++ (Sezione 6). Output: shape (K, d) .

7.2 Pesì: uniformi

```

1 self._pi = np.full(self.n_categories, 1.0 / self.n_categories)

```

`np.full(K, 1/K)` crea un vettore di K elementi tutti uguali a $1/K$. Per $K = 4$: $\pi = [0.25, 0.25, 0.25, 0.25]$.

Questa è la scelta più “agnostica”: nessuna componente è favorita a priori. I pesi verranno aggiornati dall’M-step.

7.3 Varianze: varianza globale replicata

```

1 self._sigma = np.tile(np.var(samples, axis=0),
2                        (self.n_categories, 1))

```

Passo per passo:

1. `np.var(samples, axis=0)`: calcola la varianza di ogni feature su tutti i campioni. Shape: $(d,)$.
2. `np.tile(..., (K, 1))`: replica il vettore K volte, creando una matrice (K, d) dove ogni riga è identica.

Perche la varianza globale e non casuale

Inizializzare con la varianza globale dei dati è una scelta ragionevole:

- È dell’ordine di grandezza giusto (non troppo grande, non troppo piccola).
- Ogni componente parte con la stessa “apertura”, e l’EM le specializza.
- Inizializzare con varianze casuali potrebbe creare componenti troppo strette (che catturano un solo punto) o troppo larghe (che ignorano la struttura).

8 Algoritmo EM

L’Expectation-Maximization è un algoritmo iterativo per la stima ML con dati incompleti (GDL8, slide 14).

8.1 Schema generale (dalle slide)

L’EM alterna due passi (GDL8, slide 14):

1. **E-step**: calcola $Q(\theta \mid \theta^{(t)}) = \mathbb{E}_{Z \mid X, \theta^{(t)}}[\ell_c(\theta \mid X, Z)]$
2. **M-step**: $\theta^{(t+1)} = \arg \max_{\theta} Q(\theta \mid \theta^{(t)})$

La funzione Q delle slide e il legame con il codice

Nelle slide la funzione ausiliaria Q è scritta così (GDL8, slide 25):

$$Q(\theta \mid \theta^{(t)}) = \sum_{j=1}^N \sum_{m=1}^M \underbrace{P(z_j = m \mid x_j, \theta^{(t)})}_{\text{peso}} \cdot \underbrace{\log[\pi_m^{(t)} P(x_j \mid \mu_m^{(t)}, \sigma_m^{(t)})]}_{\text{log del termine pesato}} \quad (13)$$

Questa formula contiene **due ingredienti** distinti. Vediamoli uno alla volta e colleghiamoli al codice.

Ingrediente 1: il “log del termine pesato” (`log_w` nel codice).

Dentro il $\log[\dots]$ della (13) c'è:

$$\pi_m \cdot P(x_j \mid \mu_m, \sigma_m)$$

Poiché nel GMM ogni componente genera dati gaussiani, $P(x_j \mid \mu_m, \sigma_m) = \mathcal{N}(x_j \mid \mu_m, \Sigma_m)$ (come spiegato nella Sezione 2.2). Quindi:

$$\log[\pi_m \cdot P(x_j \mid \mu_m, \sigma_m)] = \log \pi_m + \log \mathcal{N}(x_j \mid \mu_m, \Sigma_m)$$

Questo è esattamente ciò che calcola `_compute_log_weighted` (Sezione 4):

$\text{log_w}[j, m] = \log \pi_m + \log \mathcal{N}(x_j \mid \mu_m, \Sigma_m) = \text{il termine dentro il } \log[\dots] \text{ della (13)}$

Ingrediente 2: il “peso” $P(z_j = m \mid x_j, \theta^{(t)})$ (`gamma` nel codice).

Il primo fattore della doppia sommatoria è il **posterior**: “con i parametri attuali $\theta^{(t)}$, qual è la probabilità che il campione x_j sia stato generato dalla componente m ?”

Questo posterior ha un nome più corto usato universalmente nei libri di testo: si chiama **responsabilità** e si indica con γ_{jm} :

$$\gamma_{jm} \stackrel{\text{def}}{=} P(z_j = m \mid x_j, \theta^{(t)})$$

Non è una formula diversa — è solo un’**abbreviazione** per non riscrivere ogni volta $P(z_j = m \mid x_j, \theta^{(t)})$.

In sintesi, la Q delle slide diventa:

$$Q(\theta \mid \theta^{(t)}) = \sum_{j=1}^N \sum_{m=1}^M \underbrace{\gamma_{jm}}_{\text{gamma}[j, m]} \cdot \underbrace{\log[\pi_m \mathcal{N}(x_j \mid \mu_m, \Sigma_m)]}_{\text{log_w}[j, m]}$$

Slide	Simbolo breve	Codice
$P(z_j=m \mid x_j, \theta^{(t)})$	γ_{jm}	<code>gamma[j, m]</code>
$\log[\pi_m P(x_j \mid \mu_m, \sigma_m)]$	$\log w_{jm}$	<code>log_w[j, m]</code>

Per il GMM, l'E-step si riduce al calcolo delle **responsabilità** γ_{jm} , e l'M-step alle formule chiuse per aggiornare i parametri.

8.2 E-step: calcolo delle responsabilità

Come si calcola γ_{jm} a partire dalla formula delle slide

Nelle slide, $P(z_j = m | x_j, \theta^{(t)})$ è un posterior. Per calcolarlo si usa il **teorema di Bayes**:

$$P(z_j=m | x_j) = \frac{P(x_j | z_j=m) P(z_j=m)}{P(x_j)} = \frac{P(x_j | z_j=m) \pi_m}{\sum_{m'=1}^M P(x_j | z_j=m') \pi_{m'}}$$

Ora, $P(x_j | z_j=m) = \mathcal{N}(x_j | \mu_m, \Sigma_m)$ (scelta di modello), e $P(z_j=m) = \pi_m$ (il mixing coefficient). Sostituendo:

$$\gamma_{jm} = \frac{\pi_m \mathcal{N}(x_j | \mu_m, \Sigma_m)}{\sum_{m'=1}^M \pi_{m'} \mathcal{N}(x_j | \mu_{m'}, \Sigma_{m'})} \quad (14)$$

Osserva che:

- Il **numeratore** è $\pi_m \cdot \mathcal{N}(x_j | \mu_m, \Sigma_m)$, ossia *proprio* $w_{jm} = \exp(\log_w[j, m])$.
- Il **denominatore** è la somma $\sum_{m'} w_{jm'}$, cioè la somma del numeratore su tutte le componenti.

Quindi γ_{jm} è semplicemente il numeratore diviso per la somma dei numeratori:

$$\gamma_{jm} = \frac{w_{jm}}{\sum_{m'} w_{jm'}}$$

```

1 # E-step
2 log_w = self._compute_log_weighted(samples)           # (N, K)
3 a_max = np.max(log_w, axis=1)                         # (N, )
4 log_denominator = a_max + np.log(
5     np.sum(np.exp(log_w - a_max[:, None]), axis=1)
6 )                                                     # (N, )
7 gamma = np.exp(log_w - log_denominator[:, None])     # (N, K)

```

Listing 7: E-step nel codice

Corrispondenza con la formula (14):

Codice	Formula delle slide
<code>log_w[j, m]</code>	$\log(\pi_m \cdot P(x_j \mu_m, \sigma_m))$ = logaritmo del numeratore della (14)
<code>log_denominator[j]</code>	$\log \sum_{m'} \pi_{m'} P(x_j \mu_{m'}, \sigma_{m'})$ = logaritmo del denominatore (via log-sum-exp)
<code>log_w - log_denominator[:, None]</code>	$\log \gamma_{jm} = \log(\text{num}) - \log(\text{den})$ (perché $\log \frac{a}{b} = \log a - \log b$)
<code>gamma = np.exp(...)</code>	$\gamma_{jm} = P(z_j=m x_j, \theta^{(t)})$

Perché si lavora in log-scala

Non calcoliamo γ con una divisione diretta ($\frac{\text{num}}{\text{den}}$) perché i valori $\pi_m \cdot \mathcal{N}(\dots)$ possono essere numeri microscopici (es. 10^{-300}) che il computer arrotonda a 0 (underflow). Invece:

1. calcoliamo tutto in *log-scala* (dove -300 è un numero perfettamente rappresentabile);
2. usiamo il **log-sum-exp trick** (Sezione 5) per il denominatore;
3. alla fine facciamo $\exp(\log \text{num} - \log \text{den})$ per tornare alle probabilità.

Le γ_{jm} risultanti sono in $[0, 1]$ e ogni riga somma a 1.

Il codice non calcola Q esplicitamente — e va bene così

Potresti chiederti: “ma nel codice dove viene calcolata la doppia sommatoria della Q ?” La risposta è: **da nessuna parte**, e non serve.

L’unico motivo per cui la Q esiste nella teoria è giustificare *perché* le formule dell’M-step funzionano. In pratica basta calcolare γ_{jm} (E-step) e poi usarlo nelle formule chiuse dell’M-step (Sezione 8.3). La Q non viene mai valutata numericamente.

8.3 M-step: aggiornamento dei parametri

Le formule dell’M-step (GDL8, slide 27) sono:

$$N_k = \sum_{j=1}^N \gamma_{jk} \quad (15)$$

$$\pi_k^{\text{new}} = \frac{N_k}{N} \quad (16)$$

$$\mu_k^{\text{new}} = \frac{1}{N_k} \sum_{j=1}^N \gamma_{jk} x_j \quad (17)$$

$$\Sigma_k^{\text{new}} = \frac{1}{N_k} \sum_{j=1}^N \gamma_{jk} (x_j - \mu_k^{\text{new}})(x_j - \mu_k^{\text{new}})^\top \quad (18)$$

Nel caso diagonale, la (18) diventa:

$$\sigma_{k,i}^{2,\text{new}} = \frac{1}{N_k} \sum_{j=1}^N \gamma_{jk} (x_{j,i} - \mu_{k,i}^{\text{new}})^2 \quad (19)$$

Dove è finita la radice quadrata della slide?

La slide 27 di GDL8 potrebbe mostrare la formula di aggiornamento per la **deviazione standard** σ_m :

$$\sigma_m^{\text{new}} = \sqrt{\frac{1}{N_m} \sum_{j=1}^N \gamma_{jm} (x_j - \mu_m^{\text{new}})^2}$$

La formula (19) nella guida è per la **varianza** $\sigma_{k,i}^2$, cioè il quadrato della deviazione standard. La relazione è banale:

$$\sigma_m^2 = (\sigma_m)^2 \quad \Longleftrightarrow \quad \sigma_m = \sqrt{\sigma_m^2}$$

Se prendi la (19) e applichi la radice quadrata a entrambi i lati, riottiene esattamente la formula della slide. **Sono la stessa formula** — la slide mostra σ , la guida mostra σ^2 .

Perché nel codice memorizziamo σ^2 e non σ ? Perché ovunque nel modello serve la *varianza*, mai la deviazione standard:

- Nella gaussiana (3): il denominatore contiene $|\Sigma|^{1/2} = \prod_i \sigma_{k,i}^2$ sotto radice, e l’esponente contiene $\frac{(x_i - \mu_i)^2}{\sigma_{k,i}^2}$.
- Nel codice (`_compute_log_weighted`): `self._sigma` appare in divisioni (`diff**2 / self._sigma`) e in logaritmi (`np.log(self._sigma)`) — sempre come varianza.

Prendere una radice quadrata per poi riquadrarla ogni volta sarebbe inutile. Quindi il codice salta la radice e lavora direttamente con σ^2 .

```

1 # M-step
2 N_k = gamma.sum(axis=0) # (K,)
3 N_k = np.maximum(N_k, 1e-10) # floor per sicurezza
4 self._pi = N_k / N # (K,)
5 self._mu = gamma.T @ samples / N_k[:, None] # (K, d)
6 diff = samples[:, None, :] - self._mu[None, :, :] # (N, K, d)
7 self._sigma = (np.sum(gamma[:, :, None] * diff**2, axis=0)
8               / N_k[:, None]) # (K, d)
9 self._sigma = np.maximum(self._sigma, 1e-6)

```

Listing 8: M-step nel codice

8.3.1 Riga 2: conteggio soft N_k

```

1 N_k = gamma.sum(axis=0) # (K,)

```

Implementa la (15). N_k è il “numero effettivo” di campioni assegnati alla componente k . Non è un intero: è una somma di probabilità. $\sum_k N_k = N$.

8.3.2 Riga 3: floor su N_k

```

1 N_k = np.maximum(N_k, 1e-10)

```

Perche il floor su N_k

Se una componente non ha responsabilità su nessun campione ($N_k \approx 0$), la divisione per N_k nelle formule successive causerebbe divisione per zero o valori astronomici. Il floor a 10^{-10} previene questo problema. Nella pratica, se N_k è così piccolo, la componente è “morta” e i suoi parametri diventano irrilevanti.

8.3.3 Riga 4: aggiornamento pesi π_k

```

1 self._pi = N_k / N # (K,)

```

Implementa la (16). La frazione di campioni (soft) assegnata a ogni componente diventa il nuovo peso.

8.3.4 Riga 5: aggiornamento medie μ_k

```

1 self._mu = gamma.T @ samples / N_k[:, None] # (K, d)

```

Questa riga è elegante. $\text{gamma.T} @ \text{samples}$ è un prodotto matriciale:

$$(\gamma^\top X)_{k,i} = \sum_{j=1}^N \gamma_{jk} \cdot x_{j,i}$$

Shape: $(K, N) \times (N, d) = (K, d)$.

Dividendo per N_k (con broadcasting da $(K,)$ a $(K, 1)$ tramite $[:, \text{None}]$), otteniamo la media pesata (17).

8.3.5 Righe 6–8: aggiornamento varianze σ_k^2

```
1 diff = samples[:, None, :] - self._mu[None, :, :] # (N, K, d)
2 self._sigma = (np.sum(gamma[:, :, None] * diff**2, axis=0)
3                / N_k[:, None]) # (K, d)
```

Passo per passo:

1. `diff`: differenze $x_{j,i} - \mu_{k,i}^{\text{new}}$ (stesso broadcasting della Sezione 4). Shape: (N, K, d) .
2. `diff**2`: scarti al quadrato. Shape: (N, K, d) .
3. `gamma[:, :, None]`: aggiunge un asse per le features, portando γ da (N, K) a $(N, K, 1)$ per il broadcasting.
4. `gamma[:, :, None] * diff**2`: pesa ogni scarto quadrato con la responsabilità. Shape: (N, K, d) .
5. `np.sum(..., axis=0)`: somma sui campioni. Shape: (K, d) .
6. `/ N_k[:, None]`: normalizza. Shape: (K, d) .

Il risultato è esattamente la (19).

8.3.6 Riga 9: floor sulle varianze

```
1 self._sigma = np.maximum(self._sigma, 1e-6)
```

Perche il floor sulle varianze

Se una componente gaussiana collassa su un singolo punto ($\sigma_{k,i}^2 \rightarrow 0$), la densità $\mathcal{N}$ diverge a $+\infty$ e la log-likelihood esplode. Questo fenomeno si chiama **singolarità** del GMM. Il floor a 10^{-6} previene questo collasso garantendo che ogni gaussiana mantenga una larghezza minima.

8.4 Convergenza

```
1 ll = np.sum(log_denominator)
2 self._ll_history.append(ll)
3 if np.abs(ll - prev_ll) < self.epsilon:
4     break
5 prev_ll = ll
```

Listing 9: Controllo di convergenza

Il criterio di convergenza è semplice: se la variazione assoluta della log-likelihood tra due iterazioni consecutive è inferiore a $\varepsilon = 10^{-8}$, l'algoritmo si ferma.

Teorema di convergenza monotona — GDL8 slide 15

Un risultato fondamentale dell'EM è che la log-likelihood **non decresce mai**:

$$\ell(\theta^{(t+1)} | X) \geq \ell(\theta^{(t)} | X)$$

Graficamente (GDL8, slide 15), la curva della log-likelihood è monotonamente crescente. Questo non garantisce il massimo globale, ma garantisce che ogni iterazione migliora (o mantiene) la soluzione.

Nel codice, `self._ll_history` registra la storia della log-likelihood. Verificare che sia monotonamente crescente è un ottimo test di correttezza (vedi Sezione 10).

Nota su log_denominator

Nel ciclo EM, la variabile `log_denominator` calcolata nell'E-step è esattamente $\log \sum_k \pi_k \mathcal{N}(x_j | \mu_k, \Sigma_k)$ per ogni j . Quindi `np.sum(log_denominator)` è la incomplete log-likelihood $\ell(\theta^{(t)} | X)$. Non serve chiamare `log_likelihood()` separatamente: il valore è già disponibile come sottoprodotto dell'E-step.

9 BIC (Bayesian Information Criterion)

9.1 Formula

Il BIC è un criterio per la selezione del modello che bilancia la bontà di adattamento (log-likelihood) con la complessità del modello (numero di parametri):

$$\text{BIC} = \ell(\hat{\theta} | X) - \frac{p}{2} \log N \quad (20)$$

dove:

- $\ell(\hat{\theta} | X)$: log-likelihood massimizzata (incomplete).
- p : numero di parametri liberi del modello.
- N : numero di campioni.

Si sceglie il modello con il **BIC più alto** (massima likelihood penalizzata).

Attenzione alla convenzione del segno

Esistono due convenzioni per il BIC. Quella usata nel testo del midterm (e nel nostro codice) è con il segno meno davanti alla penalità, quindi si **massimizza**. In alcune fonti il BIC è definito con segno opposto e si minimizza. Controlla sempre quale convenzione è usata!

9.2 Conteggio dei parametri

Per un GMM con K componenti, d features, e covarianza diagonale:

Parametro	Conteggio per componente	Totale
π_k (pesi)	—	$K - 1$ ¹
μ_k (medie)	d per componente	$K \cdot d$
σ_k^2 (varianze diag.)	d per componente	$K \cdot d$
Totale p		$(K - 1) + 2Kd$

Nel nostro caso con $d = 5$:

K	$K - 1$	$2Kd$	$p = (K - 1) + 2Kd$
1	0	10	10
2	1	20	21
3	2	30	32
4	3	40	43
5	4	50	54
6	5	60	65

¹I pesi hanno il vincolo $\sum_k \pi_k = 1$, quindi solo $K - 1$ sono liberi.

9.3 Il codice

```
1 def bic(self, samples: np.ndarray) -> np.ndarray:
2     ll = self.log_likelihood(samples)
3     n = samples.shape[0]
4     d = samples.shape[1]
5     n_params = (self.n_categories - 1) + 2 * self.n_categories * d
6     return ll - (n_params / 2) * np.log(n)
```

Listing 10: Calcolo del BIC

- **Riga 2:** calcola $\ell(\hat{\theta}|X)$ con la funzione già analizzata.
- **Riga 5:** conta i parametri: $(K - 1) + 2Kd$.
- **Riga 6:** applica la formula (20).

9.4 Risultati del training e interpretazione

```
1 k=1  BIC=-8320.1771  logP(X|theta)=-8286.7541
2 k=2  BIC=-8052.0416  logP(X|theta)=-7981.8532
3 k=3  BIC=-7919.3844  logP(X|theta)=-7812.4306
4 k=4  BIC=-7869.8688  logP(X|theta)=-7726.1496
5 k=5  BIC=-7899.1658  logP(X|theta)=-7718.6813
6 k=6  BIC=-7916.2819  logP(X|theta)=-7699.0320
7 Best model: k=4
```

Listing 11: Output del training

Analisi dei risultati

Osservazioni:

1. La log-likelihood $\ell(\hat{\theta}|X)$ cresce monotonamente con K (da -8286.8 a -7699.0). Questo è atteso: più componenti = più flessibilità = migliore fit.
2. Il BIC cresce da $K = 1$ a $K = 4$ e poi **cala** per $K = 5$ e $K = 6$. La penalità sulla complessità supera il guadagno in likelihood.
3. Il massimo BIC si ha per $K = 4$ (BIC = -7869.87), quindi il modello selezionato ha **4 componenti**.

Interpretazione: il dataset contiene probabilmente 4 cluster naturali. Con $K = 5$ e $K = 6$ si stanno modellando rumore o sotto-strutture irrilevanti, e la penalità BIC li punisce.

Penalità BIC per $K = 4$ vs $K = 5$:

- $K = 4$: penalità = $\frac{43}{2} \log(800) = 21.5 \times 6.685 = 143.72$
- $K = 5$: penalità = $\frac{54}{2} \log(800) = 27.0 \times 6.685 = 180.49$
- La differenza di penalità (≈ 36.8) è maggiore del guadagno in log-likelihood ($-7718.7 - (-7726.1) = 7.5$), quindi $K = 5$ non è giustificato.

10 Checklist di verifica

Questa checklist permette di verificare che l'implementazione sia corretta e che il modello funzioni come atteso.

10.1 Proprietà da verificare

1. La log-likelihood cresce monotonamente

Dopo il training, verifica che `self._ll_history` sia una sequenza non decrescente:

```
diffs = np.diff(model._ll_history)
assert np.all(diffs >= -1e-6), "LL non monotona!"
```

Piccole violazioni ($\sim 10^{-10}$) sono accettabili per errori di arrotondamento. Violazioni significative indicano un bug.

2. Le righe di gamma sommano a 1

Dopo l'E-step, ogni $\gamma_{j,:}$ deve sommare a 1 (è una distribuzione di probabilità sulle componenti):

```
row_sums = gamma.sum(axis=1)
assert np.allclose(row_sums, 1.0), "Gamma rows != 1"
```

3. I pesi pi sommano a 1

I mixing coefficients devono formare una distribuzione:

```
assert np.isclose(model._pi.sum(), 1.0), "Pi sum != 1"
```

4. Le varianze sono sempre positive

Grazie al floor `np.maximum(self._sigma, 1e-6)`, le varianze non possono diventare negative o zero:

```
assert np.all(model._sigma > 0), "Varianze non positive!"
```

5. Il BIC seleziona correttamente K

Il modello con il BIC più alto deve essere quello ottimale. Nel nostro caso, $K = 4$ deve avere il BIC massimo tra tutti i modelli provati.

```
bics = [models[k].bic(train_data) for k in range(1,7)]
best_k = np.argmax(bics) + 1
assert best_k == 4, f"BIC seleziona K={best_k}, atteso K=4"
```

6. Assenza di NaN e Inf

Nessun parametro deve contenere valori non finiti:

```
assert np.all(np.isfinite(model._mu)), "NaN/Inf in mu"
assert np.all(np.isfinite(model._sigma)), "NaN/Inf in sigma"
assert np.all(np.isfinite(model._pi)), "NaN/Inf in pi"
assert np.isfinite(model.log_likelihood(train_data)), "LL non finita"
```

7. Le sezioni 3 e 4 del notebook non sono state alterate

Il testo del midterm specifica che alcune sezioni del notebook non devono essere modificate. Verificare che le celle delle sezioni 3 e 4 siano rimaste invariate rispetto all'originale.

```
# Verifica manuale: confronta le celle del notebook
# con la versione originale scaricata
```

10.2 Tabella riassuntiva

#	Proprietà	Dove verificare	Tolleranza
1	Log-likelihood monotona	<code>_ll_history</code>	10^{-6}
2	$\sum_k \gamma_{jk} = 1$	dopo E-step	10^{-10}
3	$\sum_k \pi_k = 1$	<code>_pi</code>	10^{-10}
4	$\sigma_{k,i}^2 > 0$	<code>_sigma</code>	$> 10^{-6}$
5	$\text{BIC} \rightarrow K = 4$	output training	esatto
6	No NaN/Inf	tutti i parametri	esatto
7	Sez. 3-4 invariate	notebook	esatto

Consiglio pratico per il debug

Se qualcosa non funziona, controlla in quest'ordine:

1. Stampa le shape di tutti gli array intermedi e confrontale con la tabella della Sezione 4.
2. Verifica che la log-likelihood cresca (punto 1).
3. Controlla γ (punto 2): se non somma a 1, il problema è nel log-sum-exp.
4. Controlla le varianze (punto 4): se diventano troppo piccole, il modello sta collassando.

Riepilogo dei riferimenti alle slide

Riferimento	Contenuto
GDL7 slide 9	Tabella apprendimento: struttura vs dati
GDL8 slide 3	Stessa tabella con EM evidenziato
GDL8 slide 13	Incomplete data e variabili latenti
GDL8 slide 14	Schema generale EM (E-step / M-step)
GDL8 slide 15	Convergenza monotona di EM (grafico)
GDL8 slide 18	Formula gaussiana univariata
GDL8 slides 19–20	Definizione GMM
GDL8 slide 21	Incomplete likelihood
GDL8 slide 24	Complete likelihood con variabili indicatrici
GDL8 slide 25	E-step: formula delle responsabilità
GDL8 slide 27	M-step: formule di aggiornamento